

Sistemi bazirani na znanju – Predlog Projekta

Sistem za procenu štete na motornom vozilu

Aleksa Šiljić SV50/2022

Uroš Došić SV20/2022

Novi Sad, 2025.

1. Motivacija

Procena štete na motornom vozilu je standardizovan višefazni postupak koji zahteva primenu stručnog znanja iz nekoliko oblasti. Postupak je delikatan i u realnoj situaciji uglavnom zahteva procenitelja na mestu nezgode. Motivacija za rešenje ovog problema jeste unificiranje procena štete pri saobraćajnim nezgodama i smanjenje subjektivnosti pojedinačnog procenitelja.

2. Opis problema

Prema srpskoj pravnoj i osiguravajućoj praksi, procena štete na motornom vozilu odvija se kroz četiri faze: određivanje vrednosti vozila na dan nastanka štete, određivanje vrednosti ostataka (upotrebljivih delova), utvrđivanje obima i vrednosti štete (troškovi popravke), i konačni obračun ukupne štete. Svaka od ovih faza ima jasno definisane ulazne parametre i formule — što je idealna polazna tačka za izgradnju baze pravila.

3. Metodologija rada

3.1. Kategorizacija vozila

Sistem koristi zvanične kategorije iz Pravilnika o podeli motornih i priključnih vozila:

- **M1**: Putnička vozila (osnova za standardnu amortizaciju).
- **N1**: Teretna vozila (različiti koeficijenti za radni sat i eksploataciju).
- **L1–L7**: Motocikli i četvorocikli.

3.2. Ulazi u sistem

Korisnik unosi podatke putem forme. Forma je podeljena u dve logičke celine.

Podaci o vozilu:

- Marka i model vozila (npr. Kia Ceed, VW Golf, BMW 320).
- Godina prve registracije (npr. 2015).
- Pređeni kilometri (npr. 95.000 km).
- Prosečni godišnji kilometri za datu kategoriju vozila (npr. 15.000 km/god — koristi se kao referentna vrednost).
- Nabavna cena novog vozila u trenutku prve registracije, u RSD.

- Opšte stanje vozila pre štete, ocenom 1–10.
- Tip korišćenja: privatno / taksi / rent-a-car / dostava.
- Ranije štete: da/ne; ako da, kvalitet prethodnih popravki (loš / prosečan / dobar).
- Airbag aktiviran tokom nezgode: da/ne.

Oštećeni delovi (jedan ili više redova):

- Naziv dela — odabir iz padajućeg menija (prednji branik, hauba, vrata vozača, vetrobransko staklo, itd.)
- Tip intervencije: zamena novim delom / popravka / farbanje.
- Cena rezervnog dela u RSD.
- Broj norma-sati za datu intervenciju.
- Cena rada po norma-satu u RSD.

3.3. Izlazi iz sistema

Sistem generiše dva izlaza:

- **Procenjena vrednost vozila na dan štete (RSD).** Intermedijarni rezultat koji sistem izračunava primenom niza koeficijenata amortizacije na nabavnu cenu. Prikazuje se korisniku jer je direktno potreban za donošenje finalne odluke.
- **Preporuka: totalna šteta ili ekonomska popravka.** Finalna odluka sistema. Ako su ukupni troškovi popravke veći ili jednaki vrednosti vozila umanjenoj za vrednost ostataka, proglašava se totalna šteta i izračunava iznos naknade. U suprotnom, sistem preporučuje popravku. Uz svaku odluku generiše se kratko objašnjenje koje pokazuje koje vrednosti su poređene i koji uslov je ispunjen.

4. Baza znanja

Baza znanja organizovana je kao skup DRL (Drools Rule Language) fajlova, podeljenih po fazama procene. Pravila su nastala kodifikovanjem metodologije procene koja se koristi u srpskoj osiguravajućoj praksi, kao i standardnih tehničkih normi za popravku vozila.

- `vehicle-evaluation.drl` — pravila za izračunavanje vrednosti vozila: amortizacija po starosti, korekcija prema pređenim kilometrima, uticaj opšteg stanja i načina korišćenja, umanjeње zbog ranijih loših popravki.
- `damage-assessment.drl` — pravila za izračunavanje troškova po stavkama: zamena dela, popravka, farbanje sa uračunatim materijalom.
- `total-loss.drl` — pravila za finalnu odluku: poređenje ukupnih troškova sa neto vrednošću vozila, proglašenje totalne štete ili preporuka popravke.
- `brand-depreciation.drt` — Drools rule template; generiše pravila amortizacije po marki vozila (*videti sekciju 7*).
- `fraud-cep.drl` — CEP pravila za detekciju sumnjivih obrazaca (*videti sekciju 6.5*).

Baza znanja popunjava se iz dva izvora: srpske pravne prakse i normi osiguranja (koeficijenti amortizacije, kriterijum totalne štete) i tehničkih normativa proizvođača vozila (norma-sati po tipu intervencije).

5. Primer izvršavanja

Scenario: VW Golf 7, prva registracija 2015. godine, pređeno 95.000 km (prosek za kategoriju 15.000 km/god – blago iznad proseka), nabavna cena novog vozila 25.000 EUR $\approx$ 2.937.500 RSD. Opšte stanje 6/10, privatno korišćenje, bez ranijih šteta. Šteta: zamena prednjeg branika (deo 28.000 RSD, 2 norma-sata po 3.500 RSD/h) i zamena haube (deo 45.000 RSD, 3 norma-sata po 3.500 RSD/h). Airbag nije aktiviran.

Korak	Pravilo koje se okida	Rezultat
1	Amortizacija po starosti: vozilo staro 10 god. $\rightarrow$ koeficijent $= 1 - 10 \times 0,076 = 0,24$	Faktor(STAROST, 0.24)
2	Korekcija km: pređeno $>$ prosek $\rightarrow$ koeficijent = 0,97 (-3 %)	Faktor(KM, 0.97)
3	Stanje 6/10 $\geq$ 5 $\rightarrow$ bez dodatnog umanjenja	Faktor(STANJE, 1.0)
4	Agregacija faktora: $2.937.500 \times 0,24 \times 0,97 \times 1,0$	VrednostVozila = 683.610 RSD
5	Trošak branika: $28.000 + 2 \times 3.500$	Trosak(branik, 35.000)
6	Trošak haube: $45.000 + 3 \times 3.500$	Trosak(hauba, 55.500)
7	Suma troškova: $35.000 + 55.500$	UkupniTroskovi = 90.500
8	Vozilo može da vozi $\rightarrow$ ostaci = 45 % od vrednosti	Ostaci = 307.625 RSD
9	Test totalne štete: $90.500 < 683.610 - 307.625 = 375.985$	EKONOMSKA POPRAVKA
10	BC : objašnjenje zašto nije totalna štetaBC upit jeKriticanDeo("branik_prednji"): branik $\rightarrow$ prednji_kraj $\rightarrow$ karoserija	Indikator(KRITICNA_KOMPONENTA, "branik_prednji")

Izlaz sistema: Vrednost vozila na dan štete: 683.610 RSD. Preporuka: ekonomska popravka, ukupan trošak 90.500 RSD. Objašnjenje: troškovi popravke (90.500 RSD) manji su od neto vrednosti vozila ($375.985 \text{ RSD} = 683.610 - 307.625$), stoga totalna šteta nije ispunjena.

6. Logika pravila u bazi znanja

Pravila su u nastavku data **pseudo-kodom** radi čitljivosti. Stvarna implementacija je u DRL sintaksi (uz dodatne @role/@expires anotacije gde je potrebno), ali sva logika je ovde u potpunosti opisana.

6.1. Forward chaining — Nivo 1: faktori vrednosti vozila

Prvi nivo pravila se okida nad direktno unetim podacima o vozilu i generiše *Faktor* objekte. Svaki faktor je nezavisan i kasnije se agregiraju množenjem u finalnu vrednost.

```
PRAVILO "Amortizacija po starosti – do 10 godina"
KADA postoji Vozilo v, gde je  $0 \leq v.\text{godinaStarosti} \leq 10$ 
TADA ubaci Faktor(STAROST,
                    koeficijent =  $\max(0.2, 1.0 - v.\text{godinaStarosti} \cdot 0.076)$ )
```

```

PRAVILO "Amortizacija po starosti – preko 10 godina"
KADA postoji Vozilo v, gde je v.godinaStarosti > 10
TADA ubaci Faktor(STAROST, koeficijent = 0.20) // donji prag 20%

PRAVILO "Korekcija prema pređenim km – iznad proseka"
KADA postoji Vozilo v, gde je v.kmGodisnje > v.prosecnoKmKategorije
TADA ubaci Faktor(KM, koeficijent = 0.97) // umanjenje 3%

PRAVILO "Korekcija prema pređenim km – ispod proseka"
KADA postoji Vozilo v, gde je v.kmGodisnje < 0.85 · v.prosecnoKmKategorije
TADA ubaci Faktor(KM, koeficijent = 1.03) // uvećanje 3%

PRAVILO "Eksploatacija – intenzivna upotreba (taksi, rent-a-car)"
KADA postoji Vozilo v, gde v.tipKoriscenja ∈ { TAKSI, RENT_A_CAR }
TADA ubaci Faktor(EKSPLOATACIJA, koeficijent = 0.92)

PRAVILO "Opšte stanje vozila"
KADA postoji Vozilo v, gde je v.stanje < 5
TADA ubaci Faktor(STANJE, koeficijent = 0.90)

```

6.2. Forward chaining — Nivo 2: troškovi po stavkama štete

Drugi nivo se okida kada su faktori već izračunati i kada za svaku oštećenu stavku postoji *OstecenoVozilo* činjenica. Generišu se *Trosak* objekti — jedan po stavci.

```

PRAVILO "Trošak zamene dela"
KADA postoji OstecenoVozilo d, gde je d.tipIntervencije = ZAMENA
TADA ukupno = d.cenaDela + d.normaSati · d.cenaRadaSat
    ubaci Trosak(d.naziv, ukupno, ZAMENA)

PRAVILO "Trošak popravke (bez zamene)"
KADA postoji OstecenoVozilo d, gde je d.tipIntervencije = POPRAVKA
TADA ukupno = d.normaSati · d.cenaRadaSat
    ubaci Trosak(d.naziv, ukupno, POPRAVKA)

PRAVILO "Trošak farbanja – uračunat materijal"
KADA postoji OstecenoVozilo d, gde je d.tipIntervencije = FARBANJE
TADA ukupno = (d.normaSati · d.cenaRadaSat) · 1.15 // +15% materijal
    ubaci Trosak(d.naziv, ukupno, FARBANJE)

```

6.3. Forward chaining — Nivo 3: agregacija i finalna odluka

Treći nivo se ne može aktivirati pre nego što su izračunati i faktori i svi troškovi. Tek tada se računa vrednost vozila, vrednost ostataka, i donosi se finalna preporuka.

```

PRAVILO "Agregacija vrednosti vozila"
KADA postoji Vozilo v
    skupi sve faktore  $F = \{ f_1, f_2, \dots, f_n \}$ 
TADA vrednost = v.nabavnaCena ·  $\prod f_i$ .koeficijent
    ubaci VrednostVozila(vrednost)

```

```

PRAVILO "Procena ostataka – vozilo je vozno"
KADA    postoji VrednostVozila vv
        postoji Vozilo v, gde je v.airbagAktiviran = NE
TADA    ubaci Ostaci(0.45 · vv.vrednost)

PRAVILO "Procena ostataka – vozilo nije vozno"
KADA    postoji VrednostVozila vv
        postoji Vozilo v, gde je v.airbagAktiviran = DA
TADA    ubaci Ostaci(0.25 · vv.vrednost)

PRAVILO "Totalna šteta"
KADA    postoji VrednostVozila(vrednost = V)
        postoji Ostaci(vrednost = 0)
        suma svih Trosak.iznos = T, gde je  $T \geq V - 0$ 
TADA    ubaci Odluka(TOTALNA_STETA, naknada =  $V - 0$ )

PRAVILO "Ekonomska popravka"
KADA    postoji VrednostVozila(vrednost = V)
        postoji Ostaci(vrednost = 0)
        suma svih Trosak.iznos = T, gde je  $T < V - 0$ 
TADA    ubaci Odluka(EKONOMSKA_POPRAVKA, naknada = T)

```

6.4. Backward chaining — strukturni i kompatibilni delovi

Backward chaining koristimo za **goal-driven** zaključivanje nad relacijama koje su rekurzivne ili imaju hijerarhijsku prirodu — situacije gde pre-računavanje cele tranzitivne zatvorenice ne skalira, a procedureni kod izvan engine-a ne uklapa se u rule-based pristup. U domenu procene štete identifikovali smo dva takva scenarija.

(1) Da li je oštećeni deo deo kritičnog sklopa vozila? Vozilo je modelovano kao stablo sklopova (branik → prednji_kraj → karoserija → VOZILO). Kritični sklopovi su karoserija, pogonski sklop i bezbednosni sistem. Sistem mora da prepozna kao „kritičan“ ne samo *karoseriju* direktno, već i bilo koji deo koji preko proizvoljnog broja međusklopova pripada nekom kritičnom sklopu.

(2) Da li za oštećeni deo postoji dostupna zamena? Pretragom kroz lanac kompatibilnih rezervnih delova: originalni deo, ekvivalent proizvođača, aftermarket alternativa, polovni deo iz salvage tržišta. Lanac može imati proizvoljnu dubinu — broj koraka nije unapred poznat.

```

FAKTI //statička baza znanja, učitana pri startu
DeoHijerarhija(branik_prednji, prednji_kraj)
DeoHijerarhija(hauba, prednji_kraj)
DeoHijerarhija(prednji_kraj, karoserija)
DeoHijerarhija(karoserija, VOZILO)
DeoHijerarhija(motor, pogonski_sklop)
DeoHijerarhija(pogonski_sklop, VOZILO)
DeoHijerarhija(airbag, bezbednosni_sistem)
DeoHijerarhija(bezbednosni_sistem, VOZILO)
KriticniSklop(karoserija)
KriticniSklop(pogonski_sklop)

```

```
KriticniSklop(bezbednosni_sistem)
```

```
UPIT pripadaSklopu(deo, sklop)
```

```
?- DeoHijerarhija(deo, sklop) //bazni slučaj
```

```
?- DeoHijerarhija(deo, $međusklop)  
    pripadaSklopu($međusklop, sklop) //rekurzivni korak
```

```
UPIT jeKriticanDeo(deo)
```

```
?- KriticniSklop($s)  
    pripadaSklopu(deo, $s)
```

Upiti se ne okidaju spontano kao FC pravila — pozivaju se iz drugih pravila u trenutku kada sistem postavlja *cilj* koji treba da dokaže:

```
PRAVILO "Šteta na kritičnoj komponenti → forsiraj stručnu proveru"
```

```
KADA postoji OstecenoVozilo(naziv = D)  
    jeKriticanDeo(D) //BC upit kao uslov pravila
```

```
TADA ubaci Indikator(KRITICNA_KOMPONENTA, D)  
    ubaci Eskalacija(STRUCNA_PROVERA, "Oštećen kritičan deo: " + D)
```

```
PRAVILO "Aktiviran airbag → bezbednosni sistem kompromitovan"
```

```
KADA postoji Vozilo(airbagAktiviran = DA)  
    jeKriticanDeo("airbag") //BC potvrda
```

```
TADA ubaci Faktor(BEZBEDNOST, 0.85) //dodatno umanjeње vrednosti
```

Pretraga teče unazad: cilj `jeKriticanDeo("motor")` ne nalazi direktno poklapanje sa `KriticniSklop`, pa engine pokušava drugi clause — `pripadaSklopu("motor", $s)` traži lanac kroz `DeoHijerarhija` i pronalazi `motor → pogonski_sklop`. Pošto je `pogonski_sklop` upravo registrovan kao kritičan, upit uspeva. Dodavanje novog dela (npr. `turbina → motor`) ne zahteva ažuriranje pravila — kritičnost se sama izvodi.

Lanac kompatibilnih rezervnih delova:

```
FAKTI
```

```
RezervniDeo(branik_kia_ceed_2015, dostupan = NE)
```

```
RezervniDeo(branik_kia_ceed_2016, dostupan = NE)
```

```
RezervniDeo(branik_aftermarket_X, dostupan = DA)
```

```
KompatibilanSa(branik_kia_ceed_2015, branik_kia_ceed_2016)
```

```
KompatibilanSa(branik_kia_ceed_2016, branik_aftermarket_X)
```

```
UPIT mozeBitiZamenjen(deo)
```

```
?- RezervniDeo(deo, dostupan = DA) //bazni slučaj
```

```
?- KompatibilanSa(deo, $alternativa)  
    mozeBitiZamenjen($alternativa) //rekurzivni korak
```

```
PRAVILO "Deo se ne može nabaviti → preporuči popravku umesto zamene"
```

```
KADA postoji OstecenoVozilo(naziv = D, tipIntervencije = ZAMENA)  
    ne ( mozeBitiZamenjen(D) ) // egacija nad BC upitom
```

```
TADA ubaci PromenaIntervencije(D, ZAMENA → POPRAVKA,  
    "Nema dostupnog rezervnog dela u lancu")
```

Cilj može biti zamenjen ("branik_kia_ceed_2015") uspeva tek na trećem nivou rekurzije: 2015 → 2016 → aftermarket_X (dostupan). Engine sam vodi DFS pretragu kroz relaciju KompatibilanSa; alternativa bi bila materijalizovati celokupnu tranzitivnu zatvorenicu kao FC činjenice, što ne skalira s veličinom kataloga rezervnih delova.

Generisanje objašnjenja (audit trail za osiguravajuću kuću) ostaje kao zaseban skup FC pravila koja sastavljaju tekst nakon donete odluke — to *nije* backward chaining iako se često tako naziva u literaturi, već forward-chained agregacija činjenica radi prezentacije korisniku.

6.5. CEP — detekcija sumnjivih zahteva

CEP (Complex Event Processing) je realizovan u Drools Fusion modulu. Pratimo dve klase obrazaca: gust niz zahteva istog vlasnika u kratkom periodu, i zahtev koji stiže pre isteka karenije od momenta aktiviranja polise. Treći obrazac koristi sliding window i akumulaciju.

```

DOGAĐAJ  ZahtevEvent
@role    event
@timestamp v.vremeZahteva
@expires  90 dana
polja    vlasnikId : tekst
          voziloBroj : tekst
          iznosZahteva : decimalni broj

OBRAZAC 1 „Dva zahteva istog vlasnika za različita vozila u 30 dana“
KADA      postoji z1 : ZahtevEvent(vlasnikId = X, voziloBroj = V1)
          postoji z2 : ZahtevEvent(vlasnikId = X, voziloBroj ≠ V1)
              gde se z2 desio u prozoru [1 s, 30 d] posle z1
TADA      ubaci FraudAlert(X, "Dva zahteva za različita vozila u 30 dana", PROVERA)

OBRAZAC 2 „Više od 3 zahteva za isto vozilo u 12 meseci“ (sliding window)
KADA      akumuliraj ZahtevEvent(voziloBroj = V) u prozoru window:time(12 m)
          i count(*) ≥ 3
TADA      ubaci FraudAlert(NULL, "Učestalost zahteva za vozilo " + V, ESKALACIJA)

```

7. Konfigurabilnost: rule template za marku vozila

Koeficijenti amortizacije nisu isti za sve marke vozila — npr. Kia tradicionalno amortizuje strmije u prvih pet godina od BMW-a, dok Mercedes ima blažu krivu zbog jakog sekundarnog tržišta. Umesto da pišemo zasebno DRL pravilo po marki (održavanje postaje noćna mora), koristimo **Drools rule template** mehanizam (.drt fajl + tabela parametara) koji u runtime-u generiše po jedno pravilo za svaku marku.

7.1. Tabela parametara po marki

Parametri se učitavaju iz CSV/Excel tabele brand-coefficients.csv:

marka	startKoef	godisnjiPad	minVrednost	kmKorekcija
Kia	1.00	0.085	0.15	0.96

VW	1.00	0.076	0.20	0.97
BMW	1.00	0.060	0.25	0.98
Mercedes	1.00	0.055	0.28	0.98
Dacia	1.00	0.095	0.12	0.95

7.2. Šablon pravila *brand-depreciation.drt*

Šablon definiše *oblik* pravila — placeholder-i `@{...}` se zamenjuju vrednostima iz tabele.

```
ŠABLON BrandDepreciationTemplate
parametri:  @{marka},  @{startKoef},  @{godisnjiPad},  @{minVrednost},  @{kmKorekcija}

PRAVILO "Amortizacija – @{marka} – do 10 godina"
KADA    postoji Vozilo v
        gde v.marka = "@{marka}"
        gde 0 ≤ v.godinaStarosti ≤ 10
TADA    koef = max(@{startKoef} - v.godinaStarosti · @{godisnjiPad},
                  @{minVrednost})
        ubaci Faktor(STAROST, koef)

PRAVILO "Amortizacija – @{marka} – preko 10 godina"
KADA    postoji Vozilo v
        gde v.marka = "@{marka}"
        gde v.godinaStarosti > 10
TADA    ubaci Faktor(STAROST, @{minVrednost})

PRAVILO "Korekcija km – @{marka}"
KADA    postoji Vozilo v
        gde v.marka = "@{marka}"
        gde v.kmGodisnje > v.prosecnoKmKategorije
TADA    ubaci Faktor(KM, @{kmKorekcija})
```

7.3. Generisana pravila (primer za Kia)

U fazi učitavanja KB-a, `ObjectDataCompiler` instancira šablon nad svakim redom tabele. Za red *Kia* nastaje:

```
PRAVILO "Amortizacija – Kia – do 10 godina"
KADA    postoji Vozilo v
        gde v.marka = "Kia"
        gde 0 ≤ v.godinaStarosti ≤ 10
TADA    koef = max(1.00 - v.godinaStarosti · 0.085, 0.15)
        ubaci Faktor(STAROST, koef)
```

Analogno za VW, BMW, Mercedes i Dacia. Dodavanje nove marke (npr. Tesla) svodi se na unos jednog reda u tabelu — bez ikakvih izmena u DRL kodu, bez rekompilacije. Ovaj mehanizam je naročito koristan kada poslovni korisnik (npr. aktuar) sam podešava krivu amortizacije bez programerske intervencije.

7.4. Strategija evaluacije i konflikti

Za svaki konkretan upit, samo se jedno pravilo iz template-a aktivira (ono za odgovarajuću marku), jer su uslovi $v.marka = "..."$ međusobno disjunktni. Ako vozilo dolazi sa markom van tabele, aktivira se osnovno *fallback* pravilo iz sekcije 6.1 (sa generičkim parametrima 0.076 / 0.20).

7.5. Template za politiku popravke delova

Pored amortizacije po marki vozila, sistem koristi i template mehanizam za generisanje pravila koja određuju dozvoljene intervencije nad pojedinim delovima vozila. Razlog za ovakav pristup jeste činjenica da različiti delovi imaju različita tehnička i bezbednosna ograničenja. Na primer, airbag sistem se po pravilu ne popravljia već isključivo menja, dok se kod branika ili haube može dozvoliti parcijalna reparacija u zavisnosti od stepena oštećenja.

Umesto ručnog pisanja velikog broja DRL pravila za svaki pojedinačni deo vozila, koristi se Drools rule template mehanizam (.drt + CSV tabela parametara). Time se poslovna pravila izdvajaju iz same implementacije i omogućava jednostavno proširivanje sistema bez izmene izvornog koda.

Parametri se učitavaju iz CSV/Excel tabele `repair-policy.csv`:

deo	maxPopravkaProcenat	dozvoliPopravku	kritican
airbag	0.00	false	true
vetrobran	0.40	true	true
branik_prednji	0.80	true	false
hauba	0.70	true	false
sasija	0.20	false	true

Značenje parametara:

- `maxPopravkaProcenat` — maksimalni procenat oštećenja pri kome je dozvoljena popravka umesto zamene;
- `dozvoliPopravku` — da li je popravka tehnički dozvoljena;
- `kritican` — označava da li deo pripada kritičnom sklopu vozila.

```
PRAVILO "Politika popravke – @{{deo}}"
KADA postoji OstecenoVozilo d
    gde d.naziv = "@{{deo}}"
    gde d.tipIntervencije = POPRAVKA
    gde d.procenatOstecenja > @{{maxPopravkaProcenat}}
TADA ubaci PromenaIntervencije(
    d.naziv,
    POPRAVKA → ZAMENA,
    "Stepen oštećenja prelazi dozvoljeni prag za popravku"
)
PRAVILO "Zabranjena popravka – @{{deo}}"
KADA postoji OstecenoVozilo d
    gde d.naziv = "@{{deo}}"
    gde d.tipIntervencije = POPRAVKA
    gde @{{dozvoliPopravku}} = false
TADA ubaci Eskalacija(
    STRUCNA_PROVERA,
    "Popravka dela nije dozvoljena: " + d.naziv
```

```

)
PRAVILO "Kritičan deo – @{{deo}}"
KADA postoji OstecenoVozilo d
    gde d.naziv = "@{{deo}}"
    gde @{{kritican}} = true
TADA ubaci Indikator(KRITICNA_KOMPONENTA, d.naziv)
ubaci Eskalacija(
    STRUCNA_PROVERA,
    "Oštećen kritičan deo vozila: " + d.naziv
)

```

Primer generisanog pravila za red airbag:

deo maxPopravkaProcenat dozvoliPopravku kritican

airbag 0.00 false true

engine generiše sledeća pravila:

```

PRAVILO "Politika popravke – airbag"
KADA postoji OstecenoVozilo d
    gde d.naziv = "airbag"
    gde d.tipIntervencije = POPRAVKA
    gde d.procenatOstecenja > 0.00
TADA ubaci PromenaIntervencije(
    d.naziv,
    POPRAVKA → ZAMENA,
    "Stepen oštećenja prelazi dozvoljeni prag za popravku"
)
PRAVILO "Zabranjena popravka – airbag"
KADA postoji OstecenoVozilo d
    gde d.naziv = "airbag"
    gde d.tipIntervencije = POPRAVKA
TADA ubaci Eskalacija(
    STRUCNA_PROVERA,
    "Popravka dela nije dozvoljena: airbag"
)
PRAVILO "Kritičan deo – airbag"
KADA postoji OstecenoVozilo d
    gde d.naziv = "airbag"
TADA ubaci Indikator(KRITICNA_KOMPONENTA, "airbag")
ubaci Eskalacija(
    STRUCNA_PROVERA,
    "Oštećen kritičan deo vozila: airbag"
)

```

8. Literatura

- Đorđević, D. — Procena štete na motornom vozilu.
djordjevic-lawyer.co.rs/procena-stete-na-motornom-vozilu
- Red Hat — Drools Documentation 8.x. docs.drools.org
- Drools Fusion — Complex Event Processing Guide. docs.drools.org/cep
- Drools — Rule Templates & ObjectDataCompiler. docs.drools.org/templates
- Pravilnik o obaveznom osiguranju u saobraćaju — Službeni glasnik RS.
- ISeeCars za praćenje trendova godišnjeg opadanja i drugih faktora kod template
-<https://www.iseecars.com/resale-value>